Popravni test iz predmeta
Algoritama i struktura podataka

Drugi praktični test održan 09.01.2024. Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je closed book test, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
4. Vodite računa da imate sve spremno što vam je neophodno.
5. Isključite mobilne telefone i odložite ih na katedru.
6. Ukupan broj bodova na testu je 30 bodova.
7. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
8. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
9. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2023_Test2_GrupaC_IT1g2021**
10. **Minimalni broj bodova koji se mogu osvojiti iz svakog pojedinačnog zadatka je 5.**
11. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime i prezima i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Konverzija liste u matricu susedstva*

Napisati algoritam koji će vršiti direktnu konverziju liste susedstva u matricu susedstva. Algoritam kao ulazni parametar treba da prihvati dinamički niz čvorova, a treba da rezultuje kreiranom matricom.

Dodatne napomene za rešavanje zadatka:

- U template-u je zakomentarisana metoda ***ConvertAdjacencyListToAdjacencyMatrix*** koja treba da sadrži implementaciju konverzije liste u matricu susedstva.
- U *Main* metodi obezbediti način za testiranje i proveru nove funkcionalnosti.

Zadatak 2 – *Prebrojavanje neponavljajućih vrednosti u nizu*

Napisati algoritam vremenske kompleksnosti O(n) koji će pretražiti niz celih brojeva i prebrojati sve vrednosti koje se ne ponavljaju.

Dodatne napomene za rešavanje zadatka:

- Obavezno je koristiti *Hash* tabelu koja je data u šablonu.
- Implementaciju pisati u okviru zakomentarisane metode ***public static int CountDistinctValues(int[] arr).***

Primer:

Input 1: *[5, 6, 3, 4, 3, 6, 4]*

Output 1: *1*

Input 2: *[1, 2, 3, 4, 5, 6]*

Output 2: *6*

Input 3: *[1, 1, 1, 1, 1]*

Output 3: *-1*

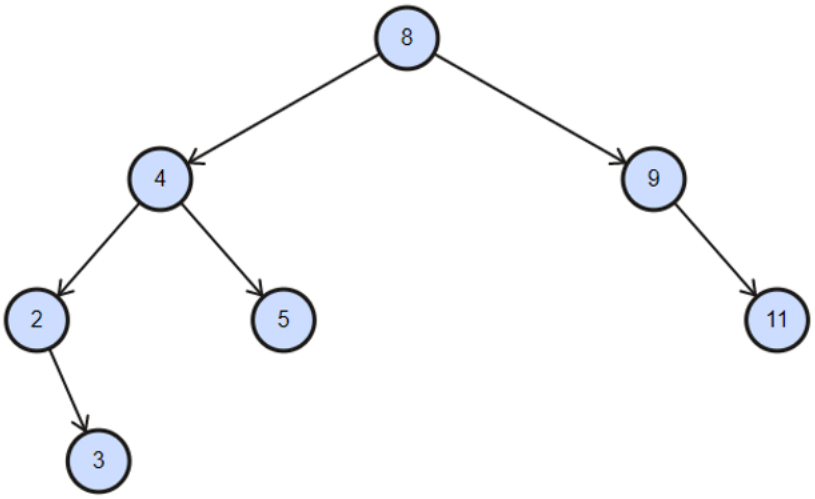
Zadatak 3 – Prebrojavanje čvorova na poziciji listova

U okviru standardne strukture klase ***AVL*** napisati implementaciju metode koja će omogućiti prebrojavanje svih čvorova koji se nalaze na poziciji listova.

Dodatne napomene za rešavanje zadatka:

- U okviru klase ***AVL*** kreirati javnu metodu sa potpisom ***public void countLeaves()***.
- Iz prethodno kreirane javne metode treba da se poziva privatna metoda sa potpisom ***private void countLeavesAVL(Node curr)***
- Privatna metoda treba da sadrži implementaciju algoritma kojim će se omogućiti prolazak kroz stablo od korena ka listovima i njihovo prebrojavanje.
- Listom se klasifikuje svaki čvor koji nema nijedno dete.
- Algoritam **OBAVEZNO** pisati u okviru klase ***AVL***

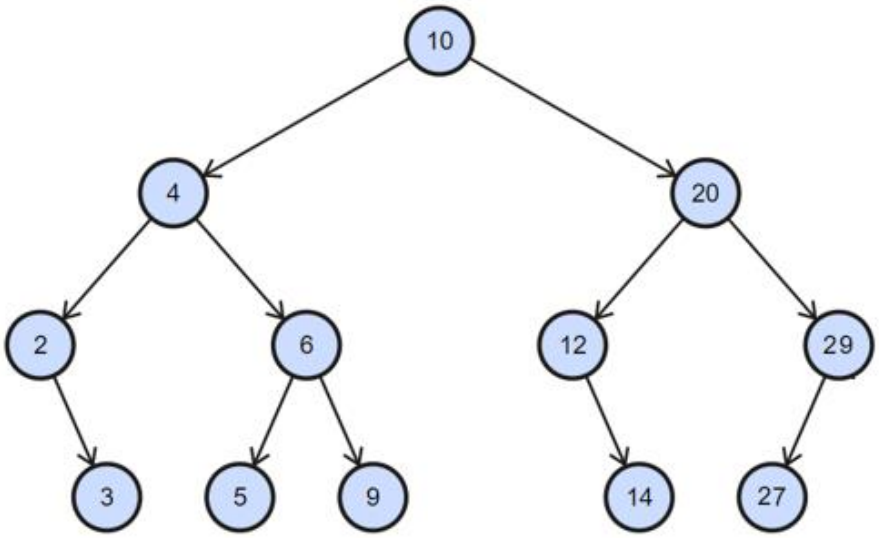
Primeri:



Slika 1 - Primer BST-a

Input 1: *BST sa slike 1*

Output 1: *3 (3, 5, 11)*



Slika 2 - Primer BST-a

Input 2: *BST sa slike 2*

Output 2: *5 (3, 5, 9, 14, 27)*